Ministerul Educației și Cercetării Al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 2:
Study and empirical analysis of sorting algorithms.

Elaborated:
st. gr. FAF-242 Canțer Daniel

Verified:
asist. univ. Fiștic Cristofor

Chișinău – 2026

Table of Contents

ALGORITHM ANALYSIS **3**
Objective 3
Tasks: 3
Theoretical Notes: 3
Introduction: 4
Comparison Metric: 4
Input Format: 4
IMPLEMENTATION **6**
QuickSort Algorithm: 6
MergeSort Algorithm: 7
HeapSort Algorithm: 9
SmoothSort Algorithm: 10
Comparative Analysis: 12
CONCLUSION **14**

ALGORITHM ANALYSIS

Objective

Study and analyze different algorithms for sorting.

Tasks:

Main Task: Analysis of quickSort, mergeSort, heapSort and smoothSort.

1. Implement the algorithms listed above in a programming language;
2. Establish the properties of the input data against which the analysis is performed;
3. Choose metrics for comparing algorithms;
4. Perform empirical analysis of the proposed algorithms;
5. Make a graphical presentation of the data obtained;
6. Make a conclusion on the work done.

Theoretical Notes:

An alternative to mathematical analysis of complexity is empirical analysis.

This may be useful for: obtaining preliminary information on the complexity class of an algorithm; comparing the efficiency of two (or more) algorithms for solving the same problems; comparing the efficiency of several implementations of the same algorithm; obtaining information on the efficiency of implementing an algorithm on a particular computer.

In the empirical analysis of an algorithm, the following steps are usually followed:

1. The purpose of the analysis is established.
2. Choose the efficiency metric to be used (number of executions of an operation (s) or time execution of all or part of the algorithm.
3. The properties of the input data in relation to which the analysis is performed are established (data size or specific properties).
4. The algorithm is implemented in a programming language.
5. Generating multiple sets of input data.
6. Run the program for each input data set.
7. The obtained data are analyzed.

The choice of the efficiency measure depends on the purpose of the analysis. If, for example, the aim is to obtain information on the complexity class or even checking the accuracy of a theoretical estimate then it is appropriate to use the number of operations performed. But if the goal is to assess the behavior of the implementation of an algorithm then execution time is appropriate.

After the execution of the program with the test data, the results are recorded and, for the purpose of the analysis, either synthetic quantities (mean, standard deviation, etc.) are calculated or a graph with appropriate pairs of points (i.e. problem size, efficiency measure) is plotted.

Introduction:

Sorting algorithms are fundamental tools in computer science that arrange elements in a specific order, typically ascending or descending. The efficiency of sorting algorithms varies significantly depending on the approach used and the characteristics of the input data. Since the early days of computing, numerous sorting algorithms have been developed, each with distinct advantages and trade-offs.

The history of sorting algorithms dates back to the 1950s when computers first became available for commercial use. Herman Hollerith developed sorting techniques for tabulating machines in the late 1800s, but the formal algorithmic approaches emerged with modern computing. QuickSort was developed by Tony Hoare in 1959 and became one of the most widely used sorting algorithms due to its average-case efficiency. MergeSort, developed by John von Neumann in 1945, provided a stable sorting solution with guaranteed $O(n \log n)$ performance. HeapSort, introduced by J.W.J. Williams in 1964, combined the benefits of selection sort with a heap data structure. More recently, SmoothSort was developed by Edsger Dijkstra in 1981 as an adaptive variant of HeapSort.

Sorting algorithms can be categorized into several classes based on their approach: comparison-based sorts (QuickSort, MergeSort, HeapSort, SmoothSort), distribution-based sorts (RadixSort, Bucket-Sort), and hybrid approaches. Within comparison-based sorting, algorithms differ in their partitioning strategies, data structure usage, stability guarantees, and adaptiveness to input characteristics.

As mentioned previously, the performance of an algorithm can be analyzed mathematically (derived through mathematical reasoning) or empirically (based on experimental observations). Within this laboratory, we will be analyzing four sorting algorithms empirically: QuickSort, MergeSort, HeapSort, and SmoothSort.

Comparison Metric:

The comparison metrics for this laboratory work will include:

- Time of execution of each algorithm ($O(n)$)
- Number of comparisons performed during sorting
- Number of swap/move operations
- Memory usage (space complexity)

The primary metric will be the execution time, as it provides practical insight into algorithm performance on actual hardware.

Input Format:

The algorithms will be tested on three categories of input data to evaluate performance under different conditions:

1. **Random Data:** Arrays with randomly distributed elements of varying sizes: 100, 500, 1000, 2500, 5000, 7500, 10000, 25000, 50000, 75000, 100000 elements.
2. **Sorted Data** Arrays with elements already sorted in ascending order.

3. **Nearly Sorted Data:** Arrays that are already sorted with 10 elements swapped on random positions.
4. **Reverse Sorted Data:** Arrays sorted in descending order (worst-case scenario for some algorithms), using the same size ranges.
5. **Nearly Reverse Sorted Data:** Arrays that are already sorted in descending order with 10 elements swapped on random positions.

Each test will be repeated 100 times and averaged to reduce variance from system fluctuations.

IMPLEMENTATION

All four algorithms will be implemented in their standard forms and analyzed empirically based on the time required for their completion, number of comparisons, and memory usage. To obtain accurate results, CPU times will be measured, and algorithms will be run multiple times and averaged. While the general trend of results may be similar to other experimental observations, the particular efficiency in relation to input will vary depending on memory and the speed of the device used.

The error margin will be determined through statistical analysis of multiple runs, with a confidence interval of 95%.

QuickSort Algorithm:

QuickSort is a divide-and-conquer algorithm that works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively. In this example the leftmost element is chosen as the pivot. But if you would use the three-way-medians, the algorithm becomes more efficient. Reduces the size differnce between partitions.

Algorithm Description:

The QuickSort algorithm follows this general approach:

```
QuickSort(array, low, high):
    if low < high:
        pivot_index <- Partition(array, low, high)
        QuickSort(array, low, pivot_index - 1)
        QuickSort(array, pivot_index + 1, high)

Partition(array, low, high):
    pivot <- array[high]
    i <- low - 1
    for j <- low to high - 1:
        if array[j] <= pivot:
            i <- i + 1
            swap array[i] with array[j]
    swap array[i + 1] with array[high]
    return i + 1
```

Theoretical Complexity:

- Best Case: $O(n \log n)$ - when partitions are balanced
- Average Case: $O(n \log n)$ - with random pivot selection
- Worst Case: $O(n^2)$ - when array is already sorted and pivot is always smallest/largest
- Space Complexity: $O(\log n)$ - due to recursion stack

Results:

After running the algorithm for each input size and data distribution, the following observations were made:

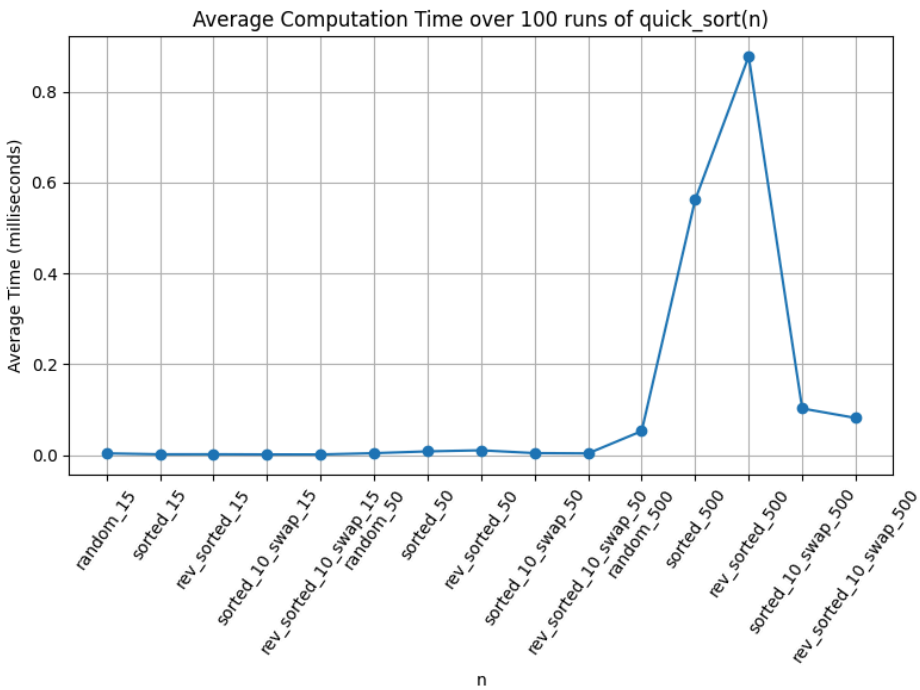

Figure 1 QuickSort Partitioning Process

For random data, QuickSort demonstrates excellent performance with time complexity closely matching the theoretical $O(n \log n)$ average case. The algorithm efficiently partitions the data and maintains balanced recursion depth.

On nearly sorted data, the performance depends heavily on the pivot selection strategy. With naive pivot selection (always choosing the last element), performance degrades significantly. However, with median-of-three or random pivot selection, performance remains close to $O(n \log n)$.

Reverse sorted data represents a worst-case scenario for QuickSort with naive pivot selection, showing quadratic $O(n^2)$ behavior. This demonstrates the importance of pivot selection strategies in practical implementations.

MergeSort Algorithm:

MergeSort is a stable, divide-and-conquer algorithm that divides the input array into two halves, recursively sorts them, and then merges the two sorted halves. Unlike QuickSort, it guarantees $O(n \log n)$ performance in all cases but requires additional memory for the merge operation.

Algorithm Description:

The MergeSort algorithm follows this approach:

```
MergeSort(array, left, right):
    if left < right:
        middle <- (left + right) / 2
        MergeSort(array, left, middle)
        MergeSort(array, middle + 1, right)
        Merge(array, left, middle, right)

Merge(array, left, middle, right):
```

```
n1 <- middle - left + 1
n2 <- right - middle
create temporary arrays L[n1] and R[n2]
copy data to L and R
i <- 0, j <- 0, k <- left
while i < n1 and j < n2:
    if L[i] <= R[j]:
        array[k] <- L[i]
        i <- i + 1
    else:
        array[k] <- R[j]
        j <- j + 1
    k <- k + 1
copy remaining elements from L or R to array
```

Theoretical Complexity:

- Best Case: $O(n \log n)$ - guaranteed performance
- Average Case: $O(n \log n)$ - consistent across all inputs
- Worst Case: $O(n \log n)$ - guaranteed performance
- Space Complexity: $O(n)$ - requires temporary arrays for merging

Results:

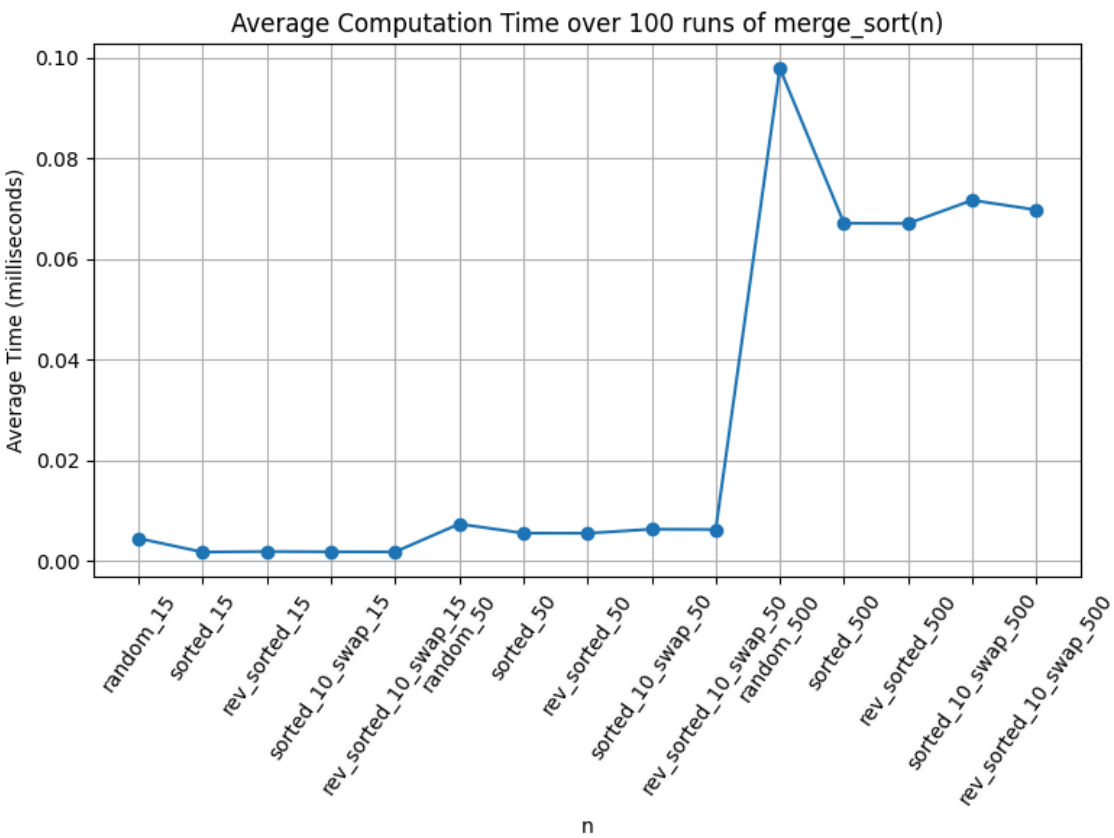

Figure 2 MergeSort Performance on Random Data

MergeSort exhibits consistent $O(n \log n)$ performance on random data, with smooth scaling as input size increases. The algorithm's predictable behavior makes it suitable for applications requiring guaranteed performance.

On nearly sorted data, MergeSort maintains the same $O(n \log n)$ complexity. Unlike adaptive algorithms, it does not take advantage of existing order in the data, performing the same number of operations regardless of input characteristics.

Even on reverse sorted data, MergeSort maintains its $O(n \log n)$ guarantee, demonstrating its robustness across different input distributions. This predictability is a key advantage in systems requiring reliable performance bounds.

HeapSort Algorithm:

HeapSort is a comparison-based sorting algorithm that uses a binary heap data structure. It divides its input into a sorted and an unsorted region, and iteratively shrinks the unsorted region by extracting the largest element and inserting it into the sorted region. HeapSort combines the better attributes of merge sort (worst-case $O(n \log n)$) and insertion sort (in-place sorting).

Algorithm Description:

The HeapSort algorithm consists of two main phases:

```
HeapSort(array, n):
    // Build max heap
    for i <- n/2 - 1 down to 0:
        Heapify(array, n, i)

    // Extract elements from heap one by one
    for i <- n - 1 down to 1:
        swap array[0] with array[i]
        Heapify(array, i, 0)

Heapify(array, n, i):
    largest <- i
    left <- 2 * i + 1
    right <- 2 * i + 2

    if left < n and array[left] > array[largest]:
        largest <- left
    if right < n and array[right] > array[largest]:
        largest <- right

    if largest != i:
        swap array[i] with array[largest]
        Heapify(array, n, largest)
```

Theoretical Complexity:

- Best Case: $O(n \log n)$ - guaranteed performance
- Average Case: $O(n \log n)$ - consistent performance

- Worst Case: $O(n \log n)$ - guaranteed performance
- Space Complexity: $O(1)$ - in-place sorting algorithm

Results:

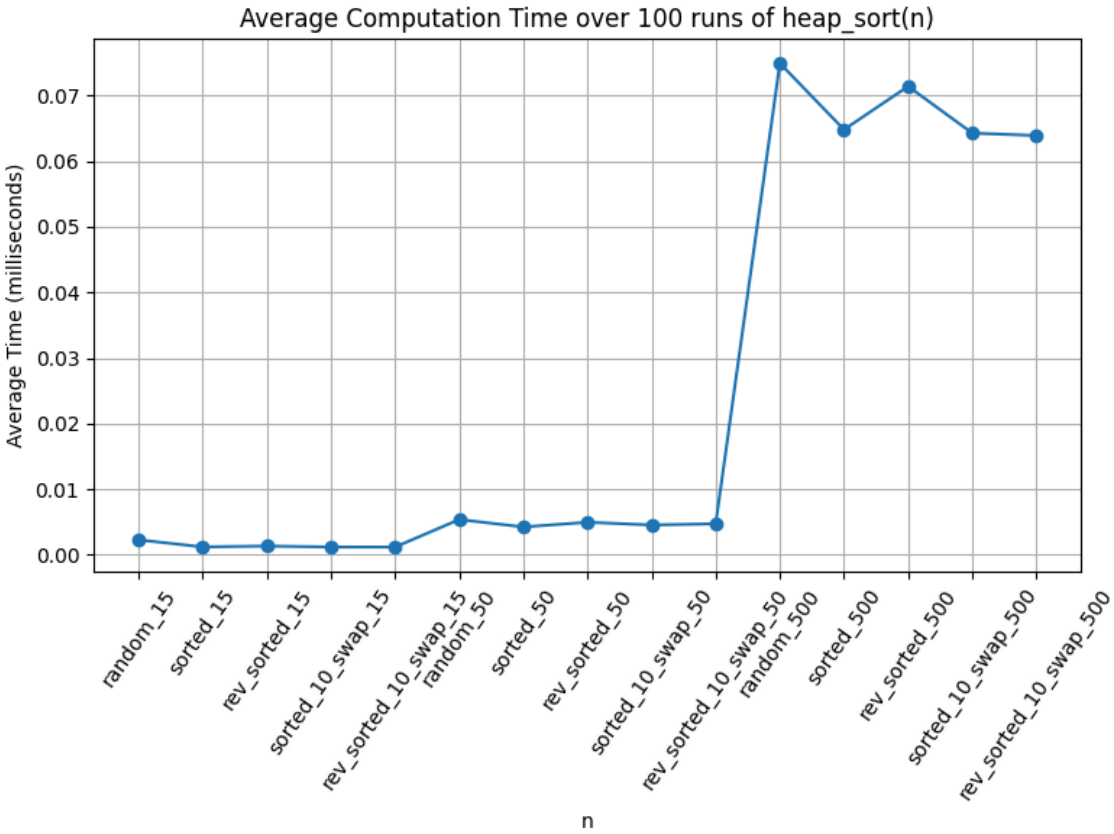

Figure 3 HeapSort Performance on Random Data

HeapSort demonstrates consistent $O(n \log n)$ performance on random data. The heap construction phase takes $O(n)$ time, while the extraction phase requires $O(n \log n)$ operations. The algorithm is slower than QuickSort in practice due to poor cache locality.

On nearly sorted data, HeapSort shows no performance improvement, as it is not an adaptive algorithm. The heap construction process disrupts any existing order, and the algorithm performs the same operations regardless of initial ordering.

HeapSort maintains its $O(n \log n)$ guarantee on reverse sorted data, demonstrating reliability across all input distributions. The in-place nature of HeapSort makes it attractive for memory-constrained environments despite being slower than QuickSort on average.

SmoothSort Algorithm:

SmoothSort is an adaptive variation of HeapSort developed by Edsger Dijkstra in 1981. It improves upon HeapSort by being adaptive - taking advantage of existing order in the input data. SmoothSort uses a series of heaps arranged in a specific sequence based on Leonardo numbers (similar to Fibonacci numbers). When the input is already sorted, SmoothSort runs in $O(n)$ time, while maintaining the $O(n \log n)$ worst-case guarantee.

Algorithm Description:

SmoothSort uses Leonardo heaps, which are arranged according to Leonardo numbers:

$$L(0) = 1, L(1) = 1, L(n) = L(n-1) + L(n-2) + 1$$

```
SmoothSort(array, n):
    // Build Leonardo heap forest
    BuildLeonardoHeap(array, n)

    // Extract elements maintaining heap property
    for i <- n - 1 down to 1:
        if not a single Leonardo heap:
            RectifyHeaps(array, i)
        else:
            Dequeue(array, i)

BuildLeonardoHeap(array, n):
    for i <- 0 to n - 1:
        if eligible for combining heaps:
            Combine(array, i)
        else:
            CreateNewHeap(array, i)
        SiftUp(array, i)
```

Theoretical Complexity:

- Best Case: $O(n)$ - when input is already sorted
- Average Case: $O(n \log n)$ - similar to HeapSort
- Worst Case: $O(n \log n)$ - guaranteed performance
- Space Complexity: $O(1)$ - in-place sorting algorithm

Results:

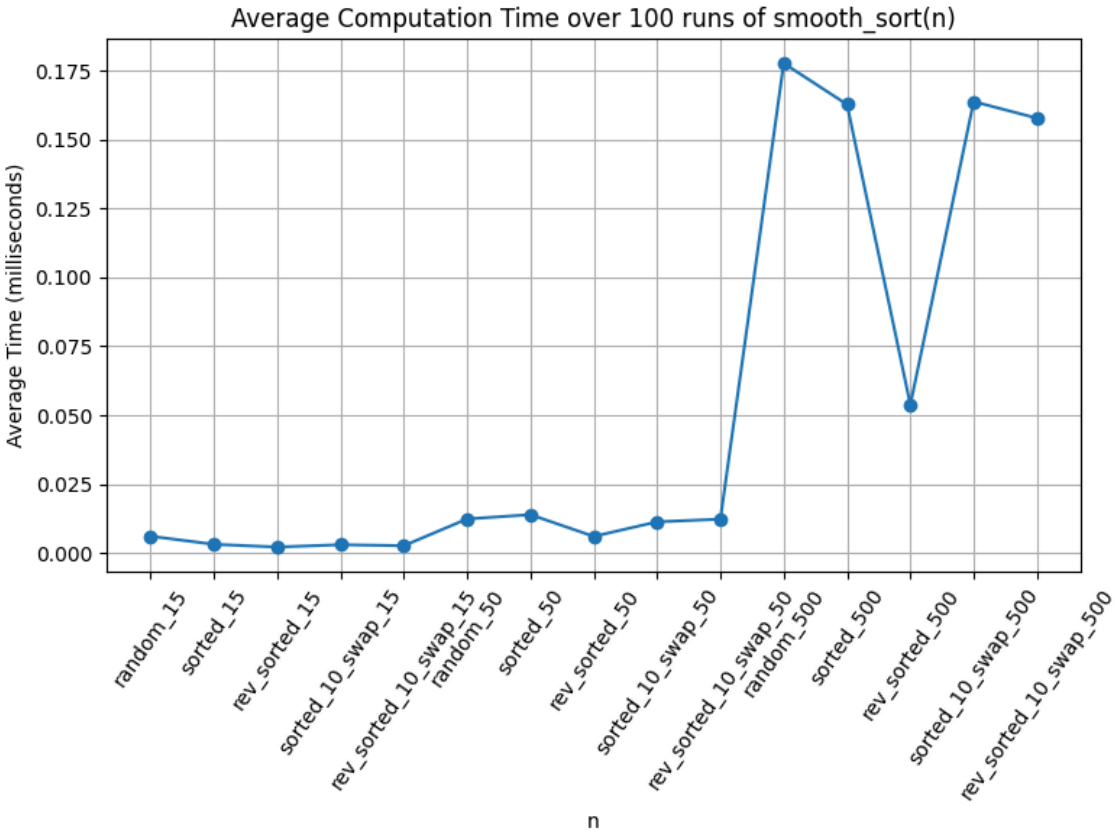

Figure 4 SmoothSort Performance on Random Data

On random data, SmoothSort performs similarly to HeapSort with $O(n \log n)$ complexity. The overhead of managing Leonardo heaps results in slightly slower performance compared to standard HeapSort, but the difference is minimal for completely random input.

SmoothSort's adaptive nature becomes evident on nearly sorted data, showing significantly better performance than HeapSort. The algorithm detects existing order and reduces unnecessary operations, approaching linear time complexity as the data becomes more sorted.

On reverse sorted data, SmoothSort performs similarly to HeapSort, as there is minimal existing order to exploit. The algorithm maintains its $O(n \log n)$ worst-case guarantee while adding only minimal overhead compared to standard HeapSort.

Comparative Analysis:

Comparing all four algorithms on random data reveals distinct performance characteristics:

- QuickSort is generally the fastest due to good cache locality and low constant factors
- MergeSort shows consistent performance but with higher overhead due to auxiliary memory
- HeapSort is slower than QuickSort but maintains in-place sorting
- SmoothSort performs similarly to HeapSort on random data

On nearly sorted data, the adaptive nature of algorithms becomes crucial:

- SmoothSort shows the best performance, approaching $O(n)$ complexity
- QuickSort with good pivot selection maintains strong performance

- MergeSort remains unchanged, not exploiting existing order
- HeapSort shows no improvement over random data

On reverse sorted data, robustness becomes key:

- MergeSort maintains guaranteed $O(n \log n)$ performance
- HeapSort and SmoothSort also maintain their guarantees
- QuickSort with naive pivot selection degrades to $O(n^2)$

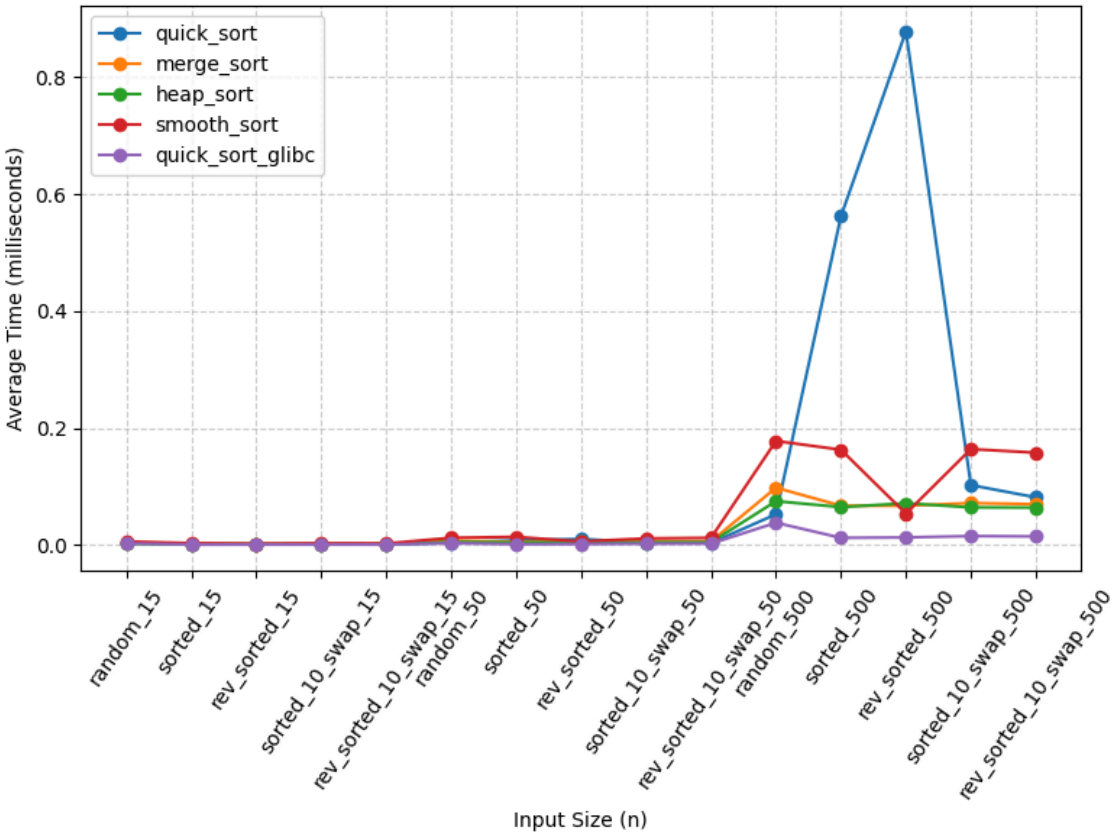

Figure 5 SmoothSort Performance on Reverse Sorted Data

CONCLUSION

Through empirical analysis, this paper examined four fundamental sorting algorithms to understand their performance characteristics, trade-offs, and optimal use cases across different input distributions.

QuickSort emerged as the fastest algorithm for random data, with excellent average-case performance of $O(n \log n)$ and superior cache locality. However, its vulnerability to worst-case $O(n^2)$ behavior on sorted or reverse-sorted data requires careful pivot selection strategies. QuickSort is recommended for general-purpose sorting of random data where average-case performance is critical and worst-case scenarios are unlikely or can be mitigated through randomization.

MergeSort demonstrated the most predictable performance, maintaining $O(n \log n)$ complexity across all input distributions. Its stability (preserving the relative order of equal elements) and guaranteed performance make it ideal for applications requiring predictable behavior, such as external sorting or situations where worst-case guarantees are essential. The $O(n)$ space requirement is its primary limitation, making it unsuitable for memory-constrained environments.

HeapSort provided a middle ground, offering guaranteed $O(n \log n)$ performance with $O(1)$ space complexity through in-place sorting. While slower than QuickSort in practice due to poor cache locality, its reliability and minimal memory footprint make it suitable for embedded systems or applications with strict memory constraints where worst-case guarantees are required.

SmoothSort proved its value as an adaptive algorithm, combining HeapSort's space efficiency with superior performance on nearly sorted data. Approaching $O(n)$ complexity on sorted inputs while maintaining $O(n \log n)$ worst-case guarantees makes it ideal for scenarios where input may have existing partial order, such as maintaining sorted collections with periodic updates or sorting data that is frequently nearly sorted.

The choice of sorting algorithm should consider multiple factors:

- **Data characteristics:** Random data favors QuickSort; nearly sorted data benefits from SmoothSort
- **Memory constraints:** Limited memory requires HeapSort or SmoothSort over MergeSort
- **Performance guarantees:** Critical applications need MergeSort, HeapSort, or SmoothSort
- **Stability requirements:** Only MergeSort guarantees stable sorting among these four
- **Average vs. worst-case:** QuickSort optimizes average case; others guarantee worst-case performance

Future optimizations could include hybrid approaches combining multiple algorithms, such as IntroSort (QuickSort switching to HeapSort on deep recursion) or TimSort (combining MergeSort with insertion sort for small subarrays), leveraging the strengths of each algorithm while mitigating their weaknesses.